AION Riemann Hypothesis Programme

Formal Mathematics Research Record, Evidence Review, and Roadmap

Document date	15 August 2026
Programme	AION formal Riemann Hypothesis research
Repository	`/Users/kevinrobinson/dev/COMDEX`
Formal authority	Lean 4.24.0 kernel and Mathlib v4.32.1
Current stage	Compact zeta divisor complete; explicit-formula bridge open
Research status	No new RH mathematics; RH remains unsolved

Purpose of this record

This document records what AION has actually built, measured, learned, and not established. It is designed to survive handovers, prevent claim inflation, support reproducibility, and define the shortest credible route from formal proof competence to useful research on a genuine RH prerequisite.

Internal research record - evidence-first, kernel-governed

Contents

Document control iv

1 Executive summary 1

1.1 Status at a glance . . . 2

2 Mathematical target and scope 2

2.1 The Riemann zeta function . . . 2

2.2 What counts as progress . . . 3

3 Programme thesis 3

3.1 Progression of research difficulty . . . 3

4 Trusted formal laboratory 4

4.1 Authority model . . . 4

4.2 Established baseline . . . 4

4.3 Fixture smoke harness . . . 4

5 First live matched experiment 5

5.1 Design . . . 5

5.2 Measured result . . . 5

5.3 Failure taxonomy . . . 5

6 Root-cause correction and continuation achievements 6

6.1 Concrete environment defect . . . 6

6.2 Truthful experiment state . . . 6

6.3 Genuine governed memory retrieval . . . 6

6.4 Governed Lean repair routing . . . 7

6.5 Fail-closed paid execution . . . 7

7 V2 held-out experiment 8

7.1 Purpose . . . 8

7.2 Matched arms . . . 8

7.3 Precommitted scoring . . . 8

8 Evaluation methodology 9

8.1 Causal questions . . . 9

8.2 Required measurements . . . 9

8.3 Statistical limitations . . . 10

9 Roadmap toward meaningful RH work **10**
9.1 Stage gates . . . 10
9.2 Immediate next steps . . . 10
9.3 Longer-term research directions . . . 11

10 Research governance **11**
10.1 Claim ladder . . . 12
10.2 Risk register . . . 12
10.3 Operating discipline . . . 13

11 Repository and evidence map **13**
11.1 Core laboratory . . . 13
11.2 Harnesses and tests . . . 14
11.3 Evidence . . . 14

12 Open questions **14**

13 Recommended research log schema **15**

14 Final programme assessment **15**

15 Post-seal execution addendum **16**

16 Mathematics continuation: compact zeta divisors and the explicit-formula interface **17**
16.1 Nontrivial zeros and the reconstructed critical strip . . . 17
16.2 Selected Chebyshev–psi criterion . . . 17
16.3 Multiplicity as analytic vanishing order . . . 18
16.4 Compact, multiplicity-aware zeta divisor . . . 18
16.5 Pole-free compact exhaustion . . . 19
16.6 Finite zero contribution and the exact open target . . . 19
16.7 Completed specialized Perron interface . . . 21
16.7.1 Exact inversion target . . . 21
16.7.2 Pinned-library capability audit . . . 22
16.8 Remaining analytic bridge . . . 22
16.9 Formal artifact map . . . 23

A Key evidence hashes **24**

B V2 statement hashes **24**

C Reproducibility commands **25**

D Selected references **25**

Document control

Version	1.1
Status	Extended through compact-divisor and finite explicit-formula formalization
Scope	Formal RH programme only; excludes unrelated prime-search, factoring, visual probes, and Photon Algebra work
Primary evidence	Lean sources, pinned environment, immutable JSON results, SHA-256 hashes, focused tests
Interpretive authority	The Lean kernel decides proof acceptance; this report interprets experiments but cannot create mathematical truth
Change rule	Future revisions must preserve prior immutable evidence and clearly separate new measurements from retrospective interpretation

Truth boundary

AION has not solved the Riemann Hypothesis. No new theorem that advances a resolution of RH has been established. Reconstructing Mathlib theorems is evidence about proof-system competence and experimental plumbing, not original mathematics. Numerical evidence, model confidence, persuasive prose, and self-awarded scores are not proofs.

1 Executive summary

The AION RH programme now has a credible formal laboratory, a pinned Lean and Mathlib authority path, strict rejection of incomplete proofs, one immutable paid baseline experiment, genuine governed retrieval from AION's persistent knowledge store, explicit diagnostic repair routing, and a six-family v2 task packet that passes local kernel verification.

The strongest verified engineering result is not a theorem about RH. It is a research-control result: the system can now distinguish a proposed Lean proof from an accepted proof, retain only compiler-verified method metadata, expose exact provenance identifiers to experimental arms, and fail closed before a paid run if task or environment hashes do not match.

The one paid experiment completed on 14 August 2026 did not establish an AION advantage. The proposer-only arm solved 0 of 3 tasks. Each AION-labelled arm solved 1 of 3. Those AION labels were not yet causally clean: memory was a generic prompt hint, repair was ordinary compiler feedback, and one task failed because the generated source omitted the Mathlib module containing the needed theorem. The result therefore validates the live provider-to-Lean path but does not measure the intended architecture.

The continuation work corrected that defect and prepared a scientifically stronger test. The next legitimate action is an independent sealing review of the v2 packet. Only after that review should exactly one bounded paid comparison run. A delayed, source-closed reconstruction test must follow at a precommitted time boundary. Only a measurable full-AION advantage over every matched ablation would justify an architectural claim.

Verified result

Final local verification status: **PASS**. The established baseline, catalogue, generated templates, and six v2 reformulations are Lean-accepted. The scoped AION store contains six active claims, six verified evidence capsules, and zero contradictions. Focused tests pass 13 of 13. Provider/API calls during this continuation phase: 0.

1.1 Status at a glance

Capability or claim	Status	Interpretation
Pinned formal laboratory	✓	Lean and Mathlib are operational authorities
Strict no-placeholder policy	✓	`sorry`, `admit`, and local axioms are prohibited
Known zeta baseline compiles	✓	Established mathematics is reconstructed
Live provider-to-Lean experiment	✓	One paid run completed and evidence is immutable
Genuine verified-memory retrieval	✓	AION persistent knowledge retrieval is now wired
Bounded diagnostic repair records	✓	Failure classes produce auditable routes
Six-family v2 packet locally verified	✓	All hidden oracle proofs compile locally
V2 independently sealed	▪	Required before credentials can load
Delayed source-closed retention	▪	Decisive architectural test not yet run
AION superiority established	✗	Existing experiment is tied and confounded
New RH-relevant mathematics	✗	None established
RH solved	✗	No

2 Mathematical target and scope

2.1 The Riemann zeta function

For complex s with $\mathrm{Re}(s) > 1$, the Riemann zeta function is represented by the absolutely convergent Dirichlet series

$$\zeta(s) = \sum_{n=1}^{\infty} \frac{1}{n^s}.$$

It admits a meromorphic continuation to the complex plane, with a simple pole at $s = 1$. Its nontrivial zeros lie in the critical strip

$$0 < \mathrm{Re}(s) < 1.$$

The Riemann Hypothesis asserts

$$\mathrm{RH}: \qquad \zeta(s) = 0,\ 0 < \mathrm{Re}(s) < 1 \quad \Longrightarrow \quad \mathrm{Re}(s) = \frac{1}{2}.$$

This project does not treat the displayed implication as a prompt to be solved directly. The target is too deep, existing formal-library coverage is partial, and an unconstrained language model can easily produce plausible but invalid mathematics. The programme therefore decomposes the challenge into governed, machine-checkable competence stages.

2.2 What counts as progress

Four evidence classes must not be conflated:

1. **Infrastructure progress**: the toolchain, verifier, hashing, memory, repair, and experiment controls work correctly.
2. **Competence progress**: AION reconstructs or transfers established proofs more reliably, cheaply, or durably than matched controls.
3. **Formalization progress**: AION formalizes known mathematics that was missing from the local dependency chain. This can be useful and novel as engineering without being new mathematics.
4. **Mathematical progress**: a genuinely new lemma, method, bound, or theorem relevant to RH survives Lean verification and independent expert mathematical review.

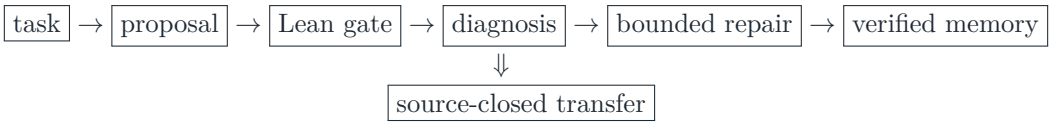
Current achievement is primarily in classes 1 and 2's experimental prerequisites. There is not yet evidence in class 4.

3 Programme thesis

The programme's central hypothesis is architectural:

> Can AION turn formally verified mathematical experience into retained, repairable competence under source closure, beyond what the same underlying proposer achieves alone?

The governed loop is



The same proposer, model version, temperature, task, tool access, token budget, attempt budget, and time budget must be held fixed. Only AION memory and repair mechanisms may vary. This is what makes an observed difference attributable to the architecture rather than to a stronger model or an easier prompt.

3.1 Progression of research difficulty

1. Known zeta and complex-analysis micro-lemmas.
2. Held-out reformulations and theorem specialization.
3. Delayed reconstruction with training sources closed.
4. Formal RH-equivalent statements and dependency mapping.
5. Complete reconstruction of a selected prerequisite chain.
6. Identification of one precise missing formalization or mathematical bottleneck.
7. Formalization and independent review of useful supporting lemmas.
8. Only then, carefully governed work on a genuinely open bottleneck.

4 Trusted formal laboratory

4.1 Authority model

The declared trusted base is the pinned Lean kernel and pinned Mathlib dependency. A candidate counts only when the Lean process returns success for the complete generated source. The harness also retains a SHA-256 source hash and compiler diagnostic.

Project-authored proof sources are scanned for prohibited incompleteness:

- `sorry`;
- `admit`;
- project-local `axiom` declarations;
- hidden placeholders or self-awarded acceptance.

This policy intentionally does not scan upstream Mathlib as though its trusted implementation were AION-authored source. The kernel return code remains the final acceptance signal.

4.2 Established baseline

The baseline reconstructs three established facts already present in Mathlib:

$$\begin{aligned} \zeta(0) &= -\frac{1}{2}, \\ \zeta(-2(n+1)) &= 0 \qquad (n \in \mathbb{N}), \\ \mathrm{Re}(s) > 1 &\Longrightarrow \zeta(s) \neq 0. \end{aligned}$$

Representative Lean source:

```
theorem reproduce_zeta_at_zero : riemannZeta 0 = -(1 : C) / 2 := by
  exact riemannZeta_zero

theorem reproduce_trivial_zero (n : N) :
    riemannZeta (-2 * ((n : C) + 1)) = 0 := by
  exact riemannZeta_neg_two_mul_nat_add_one n

theorem reproduce_zero_free_right_half_plane {s : C} (hs : 1 < s.re) :
    riemannZeta s != 0 := by
  exact riemannZeta_ne_zero_of_one_lt_re hs
```

In the actual Lean files, `C`, `N`, and `!=` above are the corresponding Unicode complex-number, natural-number, and not-equal symbols. They are rendered here in ASCII for portable documentation.

4.3 Fixture smoke harness

The fixture-only harness validates Lean compilation, proof-policy enforcement, evidence hashing, source-closed fixture reconstruction, repair recording, and claim boundaries. Its status is `PASS_TRUSTED_LAB_SMOKE_TEST`.

This is deliberately a plumbing test. Fixed candidates do not measure model quality, AION intelligence, memory advantage, repair advantage, transfer, or retention.

5 First live matched experiment

5.1 Design

One live experiment used GPT-4.1 through the OpenAI Responses API. Four arms received the same three established theorem tasks, with at most two attempts per task:

1. proposer alone;
2. AION-labelled memory plus repair;
3. AION-labelled repair without memory;
4. AION-labelled memory without repair.

The proposer returned proof bodies. The harness generated complete Lean files, rejected prohibited tokens, invoked the pinned compiler, and retained prompts, proofs, diagnostics, hashes, provider identifiers, token usage, and elapsed time.

5.2 Measured result

Table 1: Paid experiment result, 14 August 2026

Arm	Accepted	Tasks	Model calls	Success rate
Proposer alone	0	3	6	0%
AION memory + repair label	1	3	5	33.3%
AION without memory label	1	3	5	33.3%
AION without repair label	1	3	5	33.3%

Additional measurements:

- resolved provider model: `gpt-4.1-2025-04-14`;
- total provider tokens: 5,449;
- recorded wall-clock duration: 134.46 seconds;
- accepted family: only the elementary value at zero;
- immutable result hash: `5795e3faa55cf2a7466a3537ac41005056d44711eaa826c84935af318437b2e6`.

Programme decision

The experiment does not support an AION superiority claim. The three AION-labelled arms tied, and the labels did not yet correspond to the genuine durable-memory and governed-repair mechanisms. The correct conclusion is a null architectural result with useful infrastructure evidence.

5.3 Failure taxonomy

Rejected candidates exhibited several recurring failure modes:

- invented or misremembered theorem identifiers;
- Lean tactic syntax errors and invalid indentation;

- incorrect binders or explicit arguments;
- theorem conclusions that did not match the generated goal;
- unsolved goals after partial rewriting;
- repeated use of a failed branch instead of theorem discovery.

These failures show why a generic "try again" loop is inadequate. The system needs catalogue discovery, exact signature inspection, diagnostic classification, branch abandonment, and verified memory.

6 Root-cause correction and continuation achievements

6.1 Concrete environment defect

The local baseline compiled, but the live generator imported only the `RiemannZeta` module. The theorem `riemannZeta_ne_zero_of_one_lt_re` is made available through the Mathlib `Dirichlet` module. The exact live right-half-plane template therefore failed even with a known-correct proof term.

The generator now imports both required modules, and regression tests compile known-correct proofs through the exact live templates. This finding changes the interpretation of the earlier failure: at least one failure attributed to the proposer was actually an experiment-environment error.

6.2 Truthful experiment state

The original packet incorrectly remained marked `not_run` after the paid result existed. It now records the completed run, its timestamped evidence path, original sealed-packet hash, result hash, model, timestamps, and a non-superiority interpretation. The immutable result itself was not rewritten.

6.3 Genuine governed memory retrieval

A scoped adapter now uses AION's `HexCorePersistentLearningRuntime.knowledge.retrieve` mechanism. Admission is fail closed:

1. the exact theorem template is compiled by Lean;
2. the return code must be zero;
3. a 64-character source SHA-256 hash must be present;
4. only then may a verified evidence capsule and claim be ingested;
5. proof bodies are not retained in the memory object;
6. retrieval returns claim IDs, capsule IDs, evidence checksums, method metadata, and the verified source hash.

The clean scoped store currently contains:

Metric	Value
Active verified claims	6
Verified evidence capsules	6
Contradictions	0
Store revision after clean rebuild	6
Retained proof bodies	0

The six acquisition families cover the zeta value at zero, trivial zeros, right-half-plane non-vanishing, differentiability away from one, right-half-plane positivity, and finiteness of zeta zeros in compact sets.

6.4 Governed Lean repair routing

The repair controller classifies diagnostics into explicit routes:

Category	Detection signal	Required response
Unknown theorem	Unknown identifier diagnostic	Search installed Mathlib, inspect exact signature, replace invented names
Syntax failure	Unexpected token, invalid tactic, malformed command	Correct Lean syntax and return one well-indented tactic body
Type mismatch	Application mismatch, failed synthesis, coercion problem	Inspect goal and theorem types; repair arguments and coercions
Unsolved goal	Remaining goals after a partial proof	Split the goal or apply a verified theorem with a matching conclusion
Unclassified or repeated failure	No trusted route or recurring branch	Abandon the branch and construct a materially different minimal proof

Each repair record contains a stable record ID, task ID, attempt number, category, diagnostic hash, instruction, and a one-attempt retry bound. A repair-enabled arm receives the record and compiler diagnostic; a no-repair arm does not.

6.5 Fail-closed paid execution

The live harness now refuses to load an API key unless all of the following are true:

- an explicitly selected packet is marked `ready`;
- every task is source closed;
- every statement hash matches;
- the pinned Lean environment hash matches;
- the verified AION memory store is nonempty;
- the packet contains normalized tasks and matched arms.

This turns “do not run yet” from a procedural note into an executable safety property.

7 V2 held-out experiment

7.1 Purpose

The v2 packet replaces the three direct reconstruction tasks with six reformulations across six families. Each statement has been independently compiled with a known-correct oracle proof, but the proposer does not receive the proof body. Statements and environment are hash frozen.

Ref.	Family	Held-out statement
T1	Special-value transfer	$2\zeta(0) = -1$
T2	Trivial-zero specialization	$\zeta(-6) = 0$
T3	Non-vanishing contrapositive	$\zeta(s) = 0 \Rightarrow \mathrm{Re}(s) \leq 1$
T4	Analytic prerequisite	$s \neq 1 \Rightarrow \zeta$ differentiable at s
T5	Right-half-plane positivity	$x > 1 \Rightarrow \mathrm{Re}(\zeta(x)) > 0$
T6	Zero-set topology	Compact sets meet the zeta zero set in finitely many points

Packet IDs:

T1 `scaled_zeta_at_zero`

T2 `trivial_zero_at_neg_six`

T3 `zero_implies_re_le_one`

T4 `zeta_differentiable_away_from_one`

T5 `zeta_real_part_positive`

T6 `compact_has_finitely_many_zeta_zeros`

7.2 Matched arms

The precommitted packet defines:

1. proposer alone;
2. full AION memory plus repair;
3. AION repair without memory;
4. AION memory without repair;
5. a cold, isolated-state control.

Each arm receives at most two attempts per task with equal token, time, and tool budgets. Exam-time memory writes are disabled, preventing early arms or tasks from leaking answers into later conditions.

7.3 Precommitted scoring

The primary score is the number of Lean-accepted tasks per arm. Secondary measures are attempts, provider tokens, and elapsed time. A superiority claim requires the full AION arm to exceed *every* matched ablation on the primary score. A tie is not evidence of superiority.

The packet remains `local_verification_only`. It must not be changed to `ready` until an independent sealing review confirms:

- no oracle proof or answer text reaches the proposer;
- task and environment hashes match the reviewed packet;
- model and sampling parameters are frozen;
- arm order and potential order effects are addressed;
- the delayed-retention schedule is precommitted;
- output locations and immutable evidence rules are confirmed.

8 Evaluation methodology

8.1 Causal questions

The next experiment should answer four distinct questions:

Reconstruction	Can the proposer produce kernel-accepted established proofs?
Memory effect	Does verified retrieval improve success or efficiency over an identical no-memory arm?
Repair effect	Does diagnostic routing improve rejected candidates over an identical no-repair arm?
Retention effect	Does the method survive source closure and time delay on equivalent reformulations?

Immediate success without delayed retention is not sufficient evidence for the programme's main thesis. The decisive property is retained, source-closed competence.

8.2 Required measurements

For every arm, task, and attempt, retain:

- task, environment, prompt, source, and output hashes;
- model alias and resolved provider model version;
- temperature, seed if supported, token cap, and tool policy;
- exact memory claim IDs, capsule IDs, and evidence checksums supplied;
- exact repair record ID, category, diagnostic hash, and route supplied;
- candidate proof body and complete bounded compiler diagnostic;
- Lean return code and generated source hash;
- provider tokens, model calls, elapsed time, and any retries;
- failure classification and branch-abandonment decision;
- immutable result path, final result hash, and prior-result linkage.

8.3 Statistical limitations

Six tasks are still a small sample. Even a perfect 6–0 result would not prove broad mathematical intelligence. The first v2 run should be treated as a bounded mechanistic study. If promising, it should be replicated on a larger, pre-registered curriculum with multiple theorem families, multiple random task draws, and delayed tests. Confidence intervals and paired task-level analyses should replace informal percentage comparisons as the sample grows.

9 Roadmap toward meaningful RH work

9.1 Stage gates

Stage	Objective	Exit criterion	Current
0	Trusted formal laboratory	Pinned compiler, strict policy, immutable evidence	✓
1	Exact-template reconstruction	Known-correct terms compile through live generator	✓
2	Real memory and repair	Provenance IDs and repair records differ causally by arm	✓
3	Sealed immediate transfer	Full-AION advantage on precommitted held-out tasks	▪
4	Delayed retention	Fresh Lean acceptance after source closure and delay	▪
5	Critical-strip dependency reconstruction	Complete formal map of existing prerequisite chain	▪
6	Missing formalization	One useful, precise known-math gap formalized	▪
7	Genuine bottleneck lemma	New candidate survives Lean and expert review	▪
8	RH-level contribution	Independent community validation of material progress	▪

9.2 Immediate next steps

Phase A - Preserve and seal

1. Add the untracked RH laboratory and evidence policy to version control in a dedicated, reviewed commit without including secrets or mutable runtime noise.
2. Conduct an independent v2 sealing review and record reviewer, timestamp, packet hash, environment hash, and decision.
3. Freeze the exact provider model alias, resolved-version recording rule, temperature, output-token cap, attempt cap, and arm order.
4. Precommit the delayed-retention interval and equivalent task forms.

Phase B - One bounded immediate comparison

1. Change only the reviewed packet status to `ready`.

2. Launch exactly one matched run and retain a process/session identifier.
3. Allow the fail-closed preflight to validate hashes before credentials load.
4. Report every arm and failure; do not rerun to improve appearance.
5. Stop and interpret against the precommitted scoring rule.

Phase C - Delayed source-closed retention

1. Keep acquisition sources closed for the precommitted interval.
2. Present equivalent formulations with fresh declaration and prompt hashes.
3. Count only new Lean acceptance; do not replay oracle proofs.
4. Compare retention, attempts, token cost, and failure modes across arms.

Phase D - Critical-strip formal programme

1. Inventory Mathlib coverage for analytic continuation, functional equation, Euler product, zero symmetry, and zero-set topology.
2. Construct a statement-level dependency graph with exact theorem names, imports, signatures, and missing edges.
3. Reconstruct the established chain under source closure.
4. Select one missing formalization whose mathematical content is known and whose addition would make the dependency graph materially more complete.
5. Submit that formalization for independent Lean and mathematical review.

9.3 Longer-term research directions

If the competence gates pass, useful research directions include:

- formal equivalences of RH and careful comparison of their dependency costs;
- explicit zero symmetry and critical-strip localization lemmas;
- formal bridges between Dirichlet-series, Euler-product, and analytic continuation representations;
- reusable bounds needed for zero-free regions or contour arguments;
- formalization of classical lemmas absent from Mathlib;
- counterexample search for proposed intermediate lemmas;
- certified numerical bounds using Arb/FLINT as supporting evidence, never as a substitute for proof;
- automated theorem discovery over installed Mathlib declarations and exact signature retrieval;
- proof-term minimization and robustness testing under import reduction.

10 Research governance

10.1 Claim ladder

Public and internal statements should use the narrowest justified claim:

Claim	Allowed now?	Required evidence
The trusted RH lab compiles known results	Yes	Pinned Lean acceptance and source hashes
A live provider-to-Lean run completed	Yes	Immutable timestamped result and provider records
AION memory and repair are wired	Yes, narrowly	Code, local verification, provenance logs, ablation design
AION improves this proposer	No	Full arm exceeds every matched ablation under precommitment
AION retains mathematical methods	No	Delayed source-closed reconstruction
AION formalized a useful known lemma	No, not yet	New Lean source, dependency impact, independent review
AION discovered new mathematics	No	Novelty analysis, Lean proof, expert review, literature search
AION advanced RH	No	Material relevance to a recognized bottleneck and external validation
AION solved RH	No	Complete proof, formal verification, extensive independent mathematical review

10.2 Risk register

Risk	Level	Mitigation
Invented theorem names	High	Installed-library search and exact signature inspection before proposal
Import/environment mismatch	High	Compile oracle terms through exact generated templates; freeze environment hash
Answer leakage into memory	High	Store verified metadata, not proof bodies; disable exam writes; source-close tasks
Arm contamination	High	Isolated state, immutable packets, logged retrieval IDs, fixed order or randomized precommitment
Repeated paid reruns	High	Exactly one bounded run; fail-closed status gate; immutable reporting
Claim inflation from ties	High	No-tie superiority rule and explicit claim matrix
Small-sample overinterpretation	Medium	Treat v2 as mechanistic; expand only after precommitted success
Mutable or lost evidence	High	Timestamped immutable results, hashes, version control, append-only lineage
Untracked repository work	High	Review and commit RH assets separately from unrelated dirty-tree changes

Risk	Level	Mitigation
Numerics mislabelled as proof	High	Separate certified-numerics lab and explicit non-proof label
Formal proof of irrelevant lemma	Medium	Dependency graph and external relevance review before promotion
Self-grading or circular authority	High	Kernel acceptance plus independent mathematical review for research claims

10.3 Operating discipline

The following rules are part of the scientific method, not merely workflow preferences:

- Never say a test is running unless a real process or session exists.
- Never spend API credit before all local gates pass.
- Do not repeatedly announce the same next step.
- If an edit fails, record the exact failure once, correct it, and verify.
- Do not mutate a frozen mission after observing failures.
- Do not delete or rewrite immutable results.
- Do not turn a tie, failure, or confounded result into a success claim.
- Keep consciousness, proof-of-life, and self-awareness claims outside the RH evaluation.
- Separate known-mathematics reconstruction, new formalization, and new mathematics in every report.

11 Repository and evidence map

11.1 Core laboratory

`research/riemann_lab/README.md`
: Scope, proof policy, and claim boundary.

`lean-toolchain`
: Pinned Lean toolchain.

`lakefile.toml`
: Lean project and Mathlib requirement.

`lake-manifest.json`
: Resolved dependency state.

`EstablishedZeta.lean`
: Kernel-accepted established baseline.

`CatalogProbe.lean`
: Exact theorem-name and signature probes.

`dependency_graph.json`
: RH prerequisite graph and selected critical-strip target.

`live_task_packet.json`
: Truthful record of the completed v1 run.

`live_task_packet_v3.json`
: Completed hash-frozen six-family comparison packet.

11.2 Harnesses and tests

`scripts/aion_riemann_lab.py`
: Fixture-only trusted smoke harness.

`scripts/aion_riemann_live_exam.py`
: Live matched experiment and fail-closed preflight.

`scripts/aion_riemann_verified_memory.py`
: Governed memory adapter and Lean repair controller.

`scripts/aion_riemann_local_verify.py`
: Unpaid authority audit, clean memory build, and report generation.

`backend/tests/test_aion_riemann_lab.py`
: Fourteen focused policy and regression tests.

11.3 Evidence

`live_exam_20260814T213917Z.json`
: Immutable paid result.

`live_exam_latest.json` Latest pointer with identical content hash.

`local_verification_latest.json`
: Final zero-cost local verification evidence.

`aion_verified_memory.json` Scoped six-claim verified memory store.

`AION_RIEMANN_LOCAL_VERIFICATION_2026-08-15.md`
: Human-readable continuation report.

Older visual probes, Photon Algebra drivers, and other historical scripts are outside the trusted programme and must not be silently merged into this evidence chain.

12 Open questions

The programme should explicitly track the following unresolved questions:

1. Does verified memory help when the held-out statement is a nontrivial reformulation rather than a direct theorem-name lookup?
2. Does diagnostic routing add value beyond raw compiler feedback?
3. How sensitive are results to task order and retrieved-memory ranking?
4. Should theorem discovery be a deterministic library-search tool rather than prompt-visible signature memory, or should both be separate arms?
5. How should cold AION differ operationally from proposer alone and from no-memory AION?
6. What delay is long enough to make source-closed retention meaningful?
7. What minimum task count supports a credible architectural claim?
8. Which critical-strip dependency is both missing from Mathlib and useful enough to justify formalization effort?

9. How will novelty be checked before any claim of new mathematics?
10. Which external mathematicians or formalization experts will review candidate bottleneck lemmas?

13 Recommended research log schema

Every future session should append a compact structured record with:

- date, operator, repository commit, dirty-tree status, and toolchain hash;
- objective and explicit non-objectives;
- packet hash, task hashes, arm definitions, model settings, and budget;
- process/session identifier for any live execution;
- exact commands or harness entry points used;
- local test result before paid work;
- immutable evidence paths and hashes after work;
- measured outcomes, failures, and unresolved anomalies;
- claim classification: infrastructure, competence, formalization, or mathematics;
- decision: stop, replicate, revise locally, advance, or abandon;
- next stage gate and the evidence required to pass it.

This log prevents narrative drift and makes negative results scientifically useful.

14 Final programme assessment

The AION RH project has crossed an important but preliminary threshold. It is no longer an unconstrained request for an AI to speculate about RH. It is a formal, evidence-bearing programme capable of rejecting invalid proofs, preserving negative results, recording exact provenance, and testing specific architectural hypotheses.

The current accomplishment is a trustworthy experimental substrate and a locally verified next experiment. That is meaningful progress in research method and AI-mathematics infrastructure. It is not evidence that AION is closer to proving RH in the mathematical sense.

The completed v3 result and the pending retention result determine the immediate research decision:

1. v3 produced a directional mechanism signal (3/6 for full AION versus 0/6 proposer), but missed the absolute 4/6 superiority floor; and
2. a delayed source-closed test showing whether any improvement is retained and transferred.

If both fail, the architecture must be revised before approaching deeper RH content. If both succeed under the precommitted rules, the programme earns the right to move into a complete critical-strip dependency reconstruction and one precise missing formalization. Either outcome is valuable because the claims remain aligned with evidence.

15 Post-seal execution addendum

The fresh v3 matched comparison completed with zero infrastructure failures. The primary scores were: proposer alone 0/6, full AION memory plus repair 3/6, AION without memory 0/6, AION without repair 2/6, and cold AION with isolated state 1/6. Full AION exceeded every comparator, but the precommitted absolute floor was 4/6. Consequently the superiority gate was not met and no superiority claim is supported.

The result is nevertheless diagnostically useful. Verified memory plus repair recovered a scaled special value, a specialized trivial zero, and real-axis positivity. Memory without repair recovered two tasks; deterministic repair without memory recovered none; isolated cold AION recovered one. This is a mechanism signal to replicate, not a scientific conclusion about general capability.

The local formal foundation now contains 17 kernel-checked established results. In particular, the programme reconstructs the strict upper boundary `riemannZeta s = 0 -> s.re < 1` from Mathlib's nonvanishing theorem on the closed half-plane. The remaining critical-strip blocker is precise: classify every zero with nonpositive real part as a member of the negative-even trivial zero family. This reconstruction is the next mathematical target; it is not a claim of new mathematics.

The curriculum now contains 36 governed tasks. A separate six-task retention packet is locally verified and frozen. The heartbeat `aion-rh-retention-gate` checks hourly, remains closed until 15 August 2026 at 08:00 UTC, then runs one full-AION versus isolated-cold comparison and disables itself.

Truth boundary

As of 15 August 2026: trusted formal laboratory - yes; genuine governed memory retrieval - yes; deterministic Mathlib discovery - yes; 17-theorem foundation - yes; full AION led all v3 controls - yes; precommitted AION superiority - no; delayed retention - scheduled, not yet measured; new RH mathematics - none; Riemann Hypothesis solved - no.

16 Mathematics continuation: compact zeta divisors and the explicit-formula interface

This section records the mathematical continuation completed after the v3 experiment. It supersedes the earlier forward-looking statements that named left-half-plane classification or compact divisor construction as future work; those established prerequisites have now been reconstructed in Lean. The section is deliberately explicit about the boundary between what the kernel has accepted and what remains an open analytic proof obligation.

The implementation recorded here is repository commit `651f65f8`. The complete Lean library builds successfully against Lean 4.24.0 and pinned Mathlib v4.32.1. The research modules contain 55 kernel-checked theorems, or 58 declarations when the three baseline reproduction checks are included. No project-authored `sorry`, `admit`, axiom, or unsafe declaration is used in the new modules.

Truth boundary

This continuation formalizes established zeta-function infrastructure and states the next explicit-formula theorem precisely. It does not prove the explicit formula, improve a zero-free region, constrain a previously unknown zero, or establish the Riemann Hypothesis. Its classification is *formalization progress*, not new mathematical progress on RH.

16.1 Nontrivial zeros and the reconstructed critical strip

The local formalization distinguishes the negative-even trivial zeros from all other zeta zeros. In mathematical notation,

$$\begin{aligned} \mathrm{Triv}(s) &:\Longleftrightarrow \exists n \in \mathbb{N}, \quad s = -2(n+1), \\ \mathrm{Nontriv}(s) &:\Longleftrightarrow \zeta(s) = 0 \quad \text{and} \quad \neg\,\mathrm{Triv}(s). \end{aligned}$$

The kernel-checked classification theorem reconstructs the established left-half-plane result

$$\zeta(s) = 0 \ \wedge\ \mathrm{Re}(s) \le 0 \quad \Longrightarrow \quad \mathrm{Triv}(s).$$

Together with nonvanishing on the closed right half-plane, this yields

$$\mathrm{Nontriv}(s) \quad \Longrightarrow \quad 0 < \mathrm{Re}(s) < 1.$$

The proof uses the completed functional equation, nonvanishing of the reflected point $1 - s$, and the exact zero classification of Mathlib's real Gamma factor. It therefore reconstructs classical mathematics rather than assuming the critical strip as a local axiom.

The standard symmetries are also checked:

$$\rho \longmapsto 1 - \rho, \qquad \rho \longmapsto \overline{\rho},$$

and hence every nontrivial zero generates the formal four-point orbit

$$\{\rho,\ 1-\rho,\ \overline{\rho},\ 1-\overline{\rho}\}\,.$$

The local predicate is proved equivalent to Mathlib's formal statement of `RiemannHypothesis`.

16.2 Selected Chebyshev–psi criterion

The programme now follows an RH-equivalent Chebyshev error-bound route. Define the remainder

$$E_\psi(x) := \psi(x) - x,$$

where

$$\psi(x) = \sum_{n \le x} \Lambda(n)$$

and Λ is the von Mangoldt function. The selected square-root-scale target is

$$\exists C > 0 \ \ \forall x \ge 2, \qquad |E_\psi(x)| \le C\sqrt{x}\,(\log x)^2. \tag{1}$$

The Lean development currently verifies the finite von Mangoldt representation of E_ψ and the right-half-plane logarithmic-derivative identity

$$\sum_{n=1}^{\infty} \frac{\Lambda(n)}{n^s} = -\frac{\zeta'(s)}{\zeta(s)}, \qquad \mathrm{Re}(s) > 1. \tag{2}$$

It also records the equivalence between RH and Equation (1) as a proposition to reconstruct, not as an axiom or completed theorem.

16.3 Multiplicity as analytic vanishing order

For $s \neq 1$, zeta is analytic at s. Its zero multiplicity is represented by the analytic order

$$m(s) := \mathrm{ord}_s(\zeta) \in \mathbb{N} \cup \{\infty\}.$$

The formal development proves:

$$\begin{aligned} \mathrm{Nontriv}(\rho) &\Longrightarrow m(\rho) \neq 0, \\ s \neq 1 &\Longrightarrow (m(s) = 0 \Longleftrightarrow \zeta(s) \neq 0), \\ s \neq 1 &\Longrightarrow m(s) \neq \infty. \end{aligned}$$

The last statement is important: it justifies converting the extended natural analytic order into an ordinary finite integer multiplicity. Its Lean proof propagates finite order from $s = 2$, where zeta is nonzero, across the connected punctured plane $\mathbb{C} \setminus \{1\}$.

16.4 Compact, multiplicity-aware zeta divisor

Let $K \subset \mathbb{C}$ avoid the pole:

$$K \subseteq \mathbb{C} \setminus \{1\}.$$

The local zeta divisor is the integer-valued locally finite function

$$D_K(s) := \mathrm{div}_K(\zeta)(s).$$

For $s \in K$, the divisor value agrees with the finite analytic vanishing order. Outside K it is zero. The following properties are kernel-checked:

1. if K is compact, then $\mathrm{supp}(D_K)$ is finite;
2. $D_K(s) \ge 0$ for every s, because zeta is analytic on K;
3. for $s \in K$,
$$s \in \mathrm{supp}(D_K) \quad \Longleftrightarrow \quad \zeta(s) = 0;$$
4. every selected nontrivial zero has strictly positive integer multiplicity; and
5. for any weight $w : \mathbb{C} \to \mathbb{C}$, the multiplicity-aware sum
$$\sum_{\rho \in \mathrm{supp}(D_K)} D_K(\rho) w(\rho)$$
is a genuine finite sum.

Pinned Mathlib exposes zeta as analytic on $\mathbb{C} \setminus \{1\}$ but does not currently expose, through the imports used by this programme, a global meromorphic-continuation theorem suitable for silently crossing the pole. The explicit pole-avoidance hypothesis is therefore a formal safety condition, not merely a presentation choice.

Representative Lean declarations are:

```
noncomputable def zetaDivisorOn (K : Set C) :
    Function.locallyFinsuppWithin K Z :=
  divisor riemannZeta K

theorem zetaDivisorOn_support_finite {K : Set C}
    (hK : IsCompact K) :
    (zetaDivisorOn K).support.Finite

theorem mem_compactZetaDivisorSupport_iff_zero
    {K : Set C} (hK : IsCompact K)
    (havoid : K subset {1}^c) {s : C} (hs : s in K) :
    s in compactZetaDivisorSupport K hK <-> riemannZeta s = 0
```

As elsewhere in this report, ASCII placeholders in listings stand for the corresponding Unicode Lean symbols used in the checked source.

16.5 Pole-free compact exhaustion

For $n \in \mathbb{N}$, define

$$K_n := \overline{B}(0, n) \setminus B\left(1, \frac{1}{n+1}\right). \tag{3}$$

Each K_n is compact and excludes 1. The shrinking deleted ball is necessary because a plain expanding closed ball eventually contains the pole, where the available analytic divisor theorem cannot be applied.

The formalization proves the exhaustion property

$$s \neq 1 \quad \Longrightarrow \quad \exists n \in \mathbb{N},\ s \in K_n. \tag{4}$$

Filtering the finite divisor support by the nontrivial-zero predicate gives

$$\mathcal{Z}_n := \{\rho \in \operatorname{supp}(D_{K_n}) : \operatorname{Nontriv}(\rho)\}.$$

The sets $\mathcal{Z}_n$ are finite, and every nontrivial zero belongs to at least one of them. Thus the formal truncation cannot permanently omit an arbitrary fixed nontrivial zero.

16.6 Finite zero contribution and the exact open target

The zero kernel required by the classical explicit formula is

$$W_x(\rho) := \frac{x^\rho}{\rho}.$$

Using the compact divisor, the checked finite contribution is

$$Z_n(x) := \sum_{\rho \in \mathcal{Z}_n} D_{K_n}(\rho) \frac{x^\rho}{\rho}. \tag{5}$$

This construction is multiplicity-aware by definition; repeated zeros do not need to be represented by duplicated list entries.

For the current convention and nonintegral $x > 1$, define the elementary pole/trivial-zero correction

$$A(x) := \log(2\pi) + \frac{1}{2}\log\left(1 - x^{-2}\right). \tag{6}$$

The next analytic theorem has now been recorded exactly as `ChebyshevExplicitFormulaTarget`:

$$\forall x > 1, \quad x \notin \mathbb{N} \quad \Longrightarrow \quad Z_n(x) \longrightarrow -E_\psi(x) - A(x). \tag{7}$$

Equivalently, once convergence and conventions are justified,

$$E_\psi(x) = -\lim_{n\to\infty} Z_n(x) - A(x).$$

The nonintegrality hypothesis avoids selecting a left-, right-, or half-weight value at jump points of ψ. A later extension may introduce the standard symmetrized convention at prime-power jumps.

Open gate

Equation (7) is a typed Lean proposition, not a proved theorem. Defining $Z_n(x)$ and proving that its summands are genuine zeros does not establish convergence, justify a contour shift, or calculate the required residues. Those are the current mathematical proof obligations.

16.7 Completed specialized Perron interface

The first Perron unit has now been constructed in `RiemannLab/Tasks/PerronFormula.lean`. Define the zeta logarithmic-derivative kernel

$$\mathcal{K}_\zeta(x,s) := -\frac{\zeta'(s)}{\zeta(s)}\frac{x^s}{s} \tag{8}$$

and the equivalent von Mangoldt L-series kernel

$$\mathcal{K}_\Lambda(x,s) := \left(\sum_{n=1}^{\infty}\frac{\Lambda(n)}{n^s}\right)\frac{x^s}{s}. \tag{9}$$

For $\mathrm{Re}(s) > 1$, the previously verified logarithmic-derivative identity gives the kernel-checked equality

$$\mathcal{K}_\Lambda(x,s) = \mathcal{K}_\zeta(x,s). \tag{10}$$

For $c > 1$ and finite height T, parametrize the vertical segment by $s = c + it$. Since $ds = i\,dt$, the factor i cancels against the i in $(2\pi i)^{-1}$. The formal normalized truncations are therefore

$$P_T^\zeta(c,x) := \frac{1}{2\pi}\int_{-T}^{T}\mathcal{K}_\zeta(x,c+it)\,dt, \tag{11}$$

$$P_T^\Lambda(c,x) := \frac{1}{2\pi}\int_{-T}^{T}\mathcal{K}_\Lambda(x,c+it)\,dt. \tag{12}$$

Lean proves that every point $c + it$ remains in the half-plane of absolute convergence and hence, for every finite T,

$$c > 1 \quad\Longrightarrow\quad P_T^\Lambda(c,x) = P_T^\zeta(c,x). \tag{13}$$

The zero-height specializations are also checked:

$$P_0^\zeta(c,x) = P_0^\Lambda(c,x) = 0.$$

Representative Lean declarations are:

```
noncomputable def zetaPerronKernel (x : R) (s : C) : C :=
  (-deriv riemannZeta s / riemannZeta s) * (x : C) ^ s / s

noncomputable def truncatedZetaPerronIntegral
    (c T x : R) : C :=
  (1 / (2 * (Real.pi : C))) *
    integral t in -T..T, zetaPerronKernel x (c + t * Complex.I)

theorem truncatedVonMangoldtPerronIntegral_eq_zeta
    {c T x : R} (hc : 1 < c) :
    truncatedVonMangoldtPerronIntegral c T x =
      truncatedZetaPerronIntegral c T x
```

The listing uses portable ASCII placeholders; the checked source uses Lean's native Unicode notation and interval integral syntax.

16.7.1 Exact inversion target

For nonintegral $x > 1$, the classical Perron conclusion required by the programme is

$$\forall c > 1, \qquad \lim_{T\to\infty} P_T^\zeta(c,x) = \psi(x). \tag{14}$$

This is now represented exactly by the Lean proposition `ChebyshevPerronInversionTarget`. Lean additionally proves that the target written with P_T^{ζ} is logically equivalent to the target written with P_T^{Λ}, using Equation (13). Neither convergence statement is asserted as a theorem.

Open gate

The finite-height equality is complete; Perron inversion is not. Equality of the two integrands on $\mathrm{Re}(s) > 1$ does not justify the limit $T \to \infty$, interchange an infinite Dirichlet series with an improper integral, or settle the value at a jump of ψ.

16.7.2 Pinned-library capability audit

The dependency audit found the following usable primitives in pinned Mathlib:

Module (prefix `Mathlib.`)	Available role
`Analysis.MellinTransform`	Mellin transform, inverse Mellin transform, and vertical integrability.
`Analysis.MellinInversion`	General Mellin inversion through Fourier inversion under its stated integrability and continuity hypotheses.
`Analysis.Complex.CauchyIntegral`	Rectangle boundary integrals and Cauchy–Goursat machinery needed for later contour shifting.
`NumberTheory.LSeries.Dirichlet`	Absolute convergence of the von Mangoldt L-series for $\mathrm{Re}(s) > 1$ and its identity with $-\zeta'/\zeta$.

No specialized arithmetic Perron theorem matching Equation (14) was found. This negative result is recorded in `perron_dependency_audit_v1.json`; it prevents the programme from treating a general Mellin inversion theorem as though all hypotheses for the discontinuous arithmetic step function had already been discharged.

16.8 Remaining analytic bridge

The Perron interface and the finite-height arithmetic/zeta identification are now complete. The shortest credible path from this interface to Equation (7) is decomposed as follows.

1. **Scalar Perron kernel limit.** For $c > 0$, $y > 0$, and $y \neq 1$, prove

$$Q_T(c, y) := \frac{1}{2\pi} \int_{-T}^{T} \frac{y^{c+it}}{c + it}\, dt \longrightarrow \begin{cases} 1, & y > 1, \\ 0, & 0 < y < 1. \end{cases} \tag{15}$$

   The omitted point $y = 1$ is the half-weight discontinuity. This isolates the Fourier/Mellin convergence issue from all arithmetic-series bookkeeping.

2. **Finite Dirichlet-polynomial inversion.** Apply Equation (15) term by term to a finite sum

$$\sum_{n \leq N} \frac{\Lambda(n)}{n^s},$$

   producing the corresponding finite partial sum of $\psi(x)$.

3. **Absolute tail control.** Use absolute convergence on $\mathrm{Re}(s) = c > 1$ to control the omitted Dirichlet-series tail uniformly enough to pass from finite polynomials to Equation (14).

4. **Rectangular contour shift.** Move the vertical line from $\mathrm{Re}(s) = c$ into the left half-plane while avoiding zeros and the pole. Formal obligations include path composition, orientation, integrability, horizontal-edge bounds, and a sequence of admissible heights.

5. **Residue accounting.** Prove that the pole at $s = 1$ supplies the main term x, each nontrivial zero ρ supplies $-m(\rho)x^{\rho}/\rho$, and the remaining pole/kernel and trivial-zero contributions combine into Equation (6).

6. **Limit and ordering control.** Relate the zeros encountered by rectangular contours to the compact exhaustion K_n, prove the required convergence mode, and justify the symmetric ordering of the conditionally convergent zero contribution.

7. **RH-equivalent estimates.** Only after the explicit formula is established should the programme prove RH implies Equation (1), and conversely use the psi bound plus analytic continuation to exclude zeros with real part greater than 1/2.

The new irreducible unit is Item 1. After substituting $u = \log y$, it is a Fourier inversion problem for a one-sided exponential. The obstacle is precise: the transformed kernel behaves like $(c + it)^{-1}$ and is not absolutely integrable on the real line, so Mathlib's directly available absolute-integrability inversion theorem does not apply without an additional regularization or Dirichlet-type argument.

Programme decision

The specialized Perron representation is no longer undefined: its kernel, normalization, vertical line, arithmetic form, zeta form, and intended limit are all formal objects with checked finite-height equivalences. The next work must prove the scalar discontinuous inversion kernel; it must not be replaced by an unjustified invocation of general Mellin inversion.

16.9 Formal artifact map

Artifact	Technical role
`RiemannLab/Tasks/NontrivialZeros.lean`	Trivial/nontrivial predicates, critical-strip classification, reflection, conjugation, four-point orbit, and RH equivalence.
`RiemannLab/Tasks/CriticalLineCriterion.lean`	Chebyshev remainder, square-root-scale criterion, finite von Mangoldt sum, and zeta logarithmic-derivative bridge.
`RiemannLab/Tasks/ZetaZeroMultiplicity.lean`	Analytic zero order, analyticity at nontrivial zeros, and zero/nonzero characterization away from the pole.
`RiemannLab/Tasks/CompactZetaDivisor.lean`	Locally finite integer divisor, compact support, finite order, support/zero equivalence, nonnegativity, and finite weighted sums.
`RiemannLab/Tasks/ExplicitFormula.lean`	Pole-free compact exhaustion, finite nontrivial-zero sets, positive multiplicities, finite x^{ρ}/ρ sums, and the open convergence target.
`RiemannLab/Tasks/PerronFormula.lean`	Normalized zeta and von Mangoldt Perron kernels, finite-height vertical integrals, their exact equality on $\mathrm{Re}(s) > 1$, and the open inversion target.

Artifact	Technical role
`explicit_formula_dependency_graph_v1.json`	Machine-readable dependency state and the precommitted next proof unit.
`perron_dependency_audit_v1.json`	Pinned Mathlib capability audit, missing specialized theorem, and scalar Perron-kernel decomposition.

Verified result

The compact multiplicity-aware divisor and the specialized finite-height Perron interface are no longer blockers. The programme now has exhaustive zero sums and an exact normalized arithmetic/zeta vertical integral. The unresolved bridge begins with the scalar Perron limit, followed by series-tail control, contour shifting, residue calculation, convergence, and the two directions of the RH-equivalent Chebyshev bound.

A Key evidence hashes

Artifact	SHA-256
Immutable paid live result	`5795e3faa55cf2a7466a3537ac41005056d44711eaa826c84935af318437b2e6`
Established zeta baseline	`d121313f104e8e9d3a62fd2270e46aa6c4a7c6888ebc7d996f004fae5a25943c`
Catalogue probe	`6691c0dff28a5b761b7c4d1da63f7820cb0b57976b21979594be093e149fd5b5`
V2 packet	`2190c93a840eebabf0ad7cbfc3dbfe53a01ca32ead4f56765143d190e50d3660`
Sealed v3 packet	`1771f8b9977ce0cf1ffac73b1284361e61ae92d4f86672afe97a241a786abb6e`
Completed v3 result	`52899a504530456f1c6f90acf28a34edddc54ccbb4ce765d145c2f9166bc951d`
Pinned v2 environment	`6776200251cfd041ecda62038ae8ff02980bf1066a332289b6c96ca015485591`

The local-verification report is regenerated and therefore receives a new hash when timestamps or verified memory state are rebuilt. The timestamped paid result is the immutable historical object.

B V2 statement hashes

Ref.	Statement SHA-256
T1	`f84686a87dff3c814d79a296232fad81531194b318c4c81fe7d5aab8c9050798`
T2	`d1ca22f154945c4c26cf5f5cecfc5bf86d52d1538375d94420ea5abf7fce2760`
T3	`3c6a7aa1eede91cbd24530d477c38f750176269d1d8c08773dd2bc510fe77d31`

Ref.	Statement SHA-256
T4	`37c2b2c0f357b924d945d3c815244f0eff7affc33372d60ba839caea750cace5`
T5	`fed50dcca60493a2beedc23cb7ea0d485c1659dbe0fabdd0b8d3e698db523d3a`
T6	`6e920be25538210c2ec4850febc0c8a50e426af5c4ba743253c84b8af3611b4b`

C Reproducibility commands

Run from the COMDEX repository root:

```
.venv/bin/python scripts/aion_riemann_local_verify.py
.venv/bin/python -m pytest backend/tests/test_aion_riemann_lab.py -q
```

The v3 live comparison is complete and protected by its one-shot execution receipt. The separate retention packet remains time-gated and must not run before its recorded due time.

D Selected references

References

[1] B. Riemann, "On the Number of Prime Numbers Less Than a Given Quantity," 1859.

[2] E. C. Titchmarsh, revised by D. R. Heath-Brown, *The Theory of the Riemann Zeta-Function*, Oxford University Press.

[3] H. M. Edwards, *Riemann's Zeta Function*, Dover Publications.

[4] Lean Prover community, *Lean 4 theorem prover and reference documentation.*

[5] Mathlib community, *Mathlib: a mathematical library for Lean*, pinned in this programme at v4.32.1.

[6] AION/COMDEX, *Riemann Hypothesis Programme continuation handover, local verification report, task packets, Lean sources, and immutable JSON evidence*, 14–15 August 2026.